

Fundamentos de Teoría de la Computación

Prácticas - Parte 1 - Computabilidad y Complejidad

Práctica 1 - La máquina de Turing	3
Ejercicio 1	3
Ejercicio 2	4
Ejercicio 3	4
Ejercicio 4	5
Ejercicio 5	6
Ejercicio 6	6
Ejercicio 7	8
Práctica 2 - La Jerarquía de la Computabilidad	10
Ejercicio 1	10
Ejercicio 2	11
Ejercicio 3	11
Ejercicio 4	12
Ejercicio 5	12
Práctica 3 - Indecibilidad	14
Ejercicio 1	14
Ejercicio 2	14
Ejercicio 3	15
Ejercicio 4	15
Ejercicio 5	15
Ejercicio 6	16
Ejercicio 7	16
Práctica 4 - Las reducciones	19
Ejercicio 1	19
Ejercicio 2	20
Ejercicio 3	20
Ejercicio 4	21
Ejercicio 5	22
Práctica 5 - Tiempo polinomial y no polinomial	23
Ejercicio 1	23
Ejercicio 2	24
Ejercicio 3	25
Ejercicio 4	26
Práctica 6	28
Ejercicio 1	28
Ejercicio 2	29

Ejercicio 3	29
Ejercicio 4	30
Ejercicio 5	30
Práctica 7	31
Ejercicio 1	31
Ejercicio 2	31
Ejercicio 3	32
Ejercicio 4	32
Ejercicio 5	32
Ejercicio 6	33
Ejercicio 7	33
Ejercicio 8	34

Práctica 1 - La máquina de Turing

Ejercicio 1

Responder breve y claramente los siguientes incisos:

1. *¿En qué se diferencia un problema de búsqueda de un problema de decisión?*

Un problema de búsqueda es aquel en el cual la MT debe “buscar” una solución específica, la cual puede ser una entre una serie de respuestas válidas

Por otra parte, un problema de decisión es aquel en el cual la MT responderá sí o no (aceptará o rechazará la entrada)

2. *¿Por qué en el caso de los problemas de decisión, podemos referirnos indistintamente a problemas y lenguajes?*

Para los problemas de decisión es indistinto el uso de problema y lenguaje porque un lenguaje es el conjunto de cadenas que la MT acepta, lo cual también hace referencia al problema que resuelve la MT. Resolver un problema es decidir si la cadena está o no en el lenguaje de la MT

3. *El problema de la satisfacibilidad de las fórmulas booleanas, en su forma de decisión, es: “Dada una fórmula ϕ , ¿existe una asignación A de valores de verdad que la hace verdadera?” Enunciar el problema de búsqueda asociado.*

El problema de búsqueda asociado sería: “Dada una fórmula ϕ , responder la asignación A de valores de verdad que la hace verdadera. En caso de no existir, responder ‘no’.”

4. *Otra visión de MT es la que genera un lenguaje (visión generadora). En el caso del problema del inciso anterior, ¿qué lenguaje generaría la MT de visión generadora que resuelve el problema?*

La MT generadora generaría el lenguaje con todas las fórmulas booleanas satisfacibles: $L = \{\phi_1, \phi_2, \phi_3, \dots\}$

5. *¿Qué postula la Tesis de Church-Turing?*

La tesis de Church-Turing dice que todo dispositivo computacional físicamente realizable puede ser simulado por una MT

6. *¿Cuándo dos MT son equivalentes? ¿Y cuándo dos modelos de MT son equivalentes?*

Dos MTs son equivalentes si aceptan el mismo lenguaje (resuelven el mismo problema)

Dos modelos de MT son equivalentes si dada una MT de un modelo hay una MT equivalente en el otro

Ejercicio 2

Dado el alfabeto $\Sigma = \{0, 1\}$:

1. Obtener el conjunto Σ^* y el lenguaje incluido en Σ^* con cadenas de a lo sumo 2 símbolos.

$$\Sigma^* = \{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, \dots\}$$

$$L' \subset \Sigma^*; L' = \{\lambda, 0, 1, 00, 01, 10, 11\}$$

2. Sea el lenguaje $L = \{0^n 1^n \mid n \geq 0\}$. Obtener los lenguajes $\Sigma^* \cap L$, $\Sigma^* \cup L$ y L^c respecto de Σ^* .

$$\Sigma^* \cap L = L \setminus \{\lambda, 01, 0011, 000111, 00001111, \dots\} \text{ (porque } L \subset \Sigma^*)$$

$$\Sigma^* \cup L = \Sigma^* \text{ (porque } L \subset \Sigma^*)$$

$$L^c = \{0, 1, 00, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, \dots\}$$

Ejercicio 3

En clase se mostró una MT no determinística (MTN) que acepta las cadenas de la forma $ha^n o hbn$, con $n \geq 0$. Construir (describir la función de transición) una MT determinística (MTD) equivalente.

$$MT = (Q, \Gamma, \delta, q_0, q_A, q_R)$$

$$Q = \{q_0, q_h, q_a, q_b, q_A, q_R\}$$

q_0 : estado inicial

q_h : buscando caracter luego de la primera 'h'

q_a : buscando una 'a'

q_b : buscando una 'b'

$$\text{alfabeto } \Gamma = \{h, a, b, B\}$$

δ :

$$1. \delta(q_0, h) = (q_h, h, R)$$

$$2. \delta(q_h, a) = (q_a, a, R)$$

$$3. \delta(q_h, b) = (q_b, b, R)$$

$$4. \delta(q_a, a) = (q_a, a, R)$$

$$5. \delta(q_b, b) = (q_b, b, R)$$

$$6. \delta(q_h, B) = (q_A, B, S)$$

$$7. \delta(q_a, B) = (q_A, B, S)$$

$$8. \delta(q_b, B) = (q_A, B, S)$$

Como tabla:

	h	a	b	B
--	---	---	---	---

q_0	q_h, h, R			
q_h		q_a, a, R	q_b, b, R	q_A, B, S
q_a		q_a, a, R		q_A, B, S
q_b			q_b, b, R	q_A, B, S

Ejercicio 4

Describir la idea general de una MT con varias cintas que acepte, de la manera más eficiente posible (menor cantidad de pasos), el lenguaje $L = \{a^n b^n c^n \mid n \geq 0\}$.

En la cinta 1 se iría avanzando sobre la cadena de entrada

Si la cadena está vacía se aceptaría directamente

Para cada letra 'a' que se encuentre, se movería en la cinta 2 a la derecha insertando una 'X' como indicador de la cantidad de letras 'a' que se van encontrando

Al encontrar una letra 'b', se comenzaría a ir a la izquierda en la cinta 2, transformando las 'X' en 'Y'. Si se llega a un Blanco se rechazaría la cadena de entrada. Si al encontrar una letra 'c' todavía quedan 'X' a la izquierda del cabezal de la cinta 2, entonces se rechazaría la cadena de entrada

Por último, cuando se encuentra una letra 'c', y no se rechazó la cadena de entrada, significa que estamos al comienzo de la cinta 2, por lo que se vuelve a hacer el proceso que se realizó con las letras 'b', pero de izquierda a derecha y buscando las 'Y', cambiándolas por 'Z'. Si al final, hay alguna 'Y' restante se rechaza (y también se rechaza si antes de terminar las letras 'c' se encuentra un Blanco)

$MT = (Q, \Gamma, \delta, q_0, q_A, q_R)$

$Q = \{q_0, q_h, q_a, q_b, q_A, q_R\}$

q_0 : estado inicial

q_a : buscando una 'a'

q_b : buscando una 'b'

q_c : buscando una 'c'

alfabeto $\Gamma = \{a, b, c, B, X, Y, Z\}$

δ :

- $\delta(q_0, (B, B)) = (q_A, (B, S), (B, S))$
- $\delta(q_0, (a, B)) = (q_a, (a, R), (X, R))$
- $\delta(q_a, (a, B)) = (q_a, (a, R), (X, R))$
- $\delta(q_a, (b, B)) = (q_b, (b, S), (B, L))$
- $\delta(q_b, (b, X)) = (q_b, (b, R), (Y, L))$
- $\delta(q_b, (c, B)) = (q_c, (c, S), (B, R))$
- $\delta(q_c, (c, Y)) = (q_c, (c, R), (Z, R))$

$$8. \delta(q_c, (B, B)) = (q_A, (B, S), (B, S))$$

Ejercicio 5

Explicar cómo una MT sin el movimiento S (el no movimiento) puede simular (ejecutar) otra que sí lo tiene.

Una MT sin S puede simular a otra que sí lo tiene (es un modelo equivalente al de la MT que sí tiene S), haciendo que, cuando se necesita realizar una S, se entre en un estado especial que realice primero R, luego L (o viceversa) y en el paso en el que se realiza L, se cambia al estado que originalmente se deseaba cambiar

En una MT con S:

$$\delta(q, a) = (q', b, S)$$

En una MT sin S:

$$\delta(q, a) = (q_{aux}, b, R)$$

$$\delta(q_{aux}, y) = (q', y, L)$$

Donde 'y' es el símbolo a la derecha del símbolo donde se deseaba hacer S

De esta manera habría que tener un estado auxiliar por cada estado diferente al que se quiere retornar, y definir una entrada de la función de transición para cada estado auxiliar con cada caracter que pueda haber, donde se retorne el mismo caracter, el estado asociado a dicho estado auxiliar, y se vuelve a la posición previa. Ejemplo:

- Si se quiere realizar $\delta(q, a) = (q', b, S)$:
 - Primero se hace $\delta(q, a) = (q_{aux}, b, R)$
 - Si en la posición de la derecha hay una 'y':

$$\delta(q_{aux}, y) = (q', y, L)$$
 - Si en la posición de la derecha hay una 'j':

$$\delta(q_{aux}, j) = (q', j, L)$$
 - Y así por cada posible símbolo a la derecha
- Si se quiere realizar $\delta(q, a) = (otro, b, S)$:
 - Primero se hace $\delta(q, a) = (q_{aux_otro}, b, R)$
 - Si en la posición de la derecha hay una 'y':

$$\delta(q_{aux_otro}, y) = (q_{otro}, y, L)$$
 - Si en la posición de la derecha hay una 'j':

$$\delta(q_{aux_otro}, j) = (q_{otro}, j, L)$$
 - Y así por cada posible símbolo a la derecha

De esta manera, los estados auxiliares crecen de acuerdo a los diferentes estados y símbolos que podrían darse en la situación mostrada

Ejercicio 6

En clase se construyó una MT con 2 cintas que acepta $L = \{w \mid w \in \{a, b\}^ \text{ y } w \text{ es un palíndromo}\}$. Construir una MT equivalente con 1 cinta. Ayuda: la solución que vimos para aceptar el lenguaje de las cadenas $a^n b^n$, con $n \geq 1$, puede ser un buen punto de partida.*

$MT = (Q, \Gamma, \delta, q_0, q_A, q_R)$

$Q = \{q_0, q_{aa}, q_{ab}, q_{fa}, q_{fb}, q_r, q_i, q_A, q_R\}$

q_0 : estado inicial

q_{aa} : avanzando para buscar una 'a' al final

q_{ab} : avanzando para buscar una 'b' al final

q_{fa} : buscando 'a' al final

q_{fb} : buscando 'b' al final

q_r : retrocediendo hasta el inicio

q_i : buscando un símbolo al inicio

alfabeto $\Gamma = \{a, b, \alpha, \beta, B\}$

δ :

1. $\delta(q_0, B) = (q_A, B, S)$

2. $\delta(q_0, a) = (q_{aa}, \alpha, R)$

3. $\delta(q_0, b) = (q_{ab}, \beta, R)$

4. $\delta(q_{aa}, a) = (q_{aa}, a, R)$

5. $\delta(q_{aa}, b) = (q_{aa}, b, R)$

6. $\delta(q_{aa}, \alpha) = (q_{fa}, \alpha, L)$

7. $\delta(q_{aa}, \beta) = (q_{fa}, \beta, L)$

8. $\delta(q_{aa}, B) = (q_{fa}, B, L)$

9. $\delta(q_{ab}, a) = (q_{ab}, a, R)$

10. $\delta(q_{ab}, b) = (q_{ab}, b, R)$

11. $\delta(q_{ab}, \alpha) = (q_{fb}, \alpha, L)$

12. $\delta(q_{ab}, \beta) = (q_{fb}, \beta, L)$

13. $\delta(q_{ab}, B) = (q_{fb}, B, L)$

14. $\delta(q_{fa}, a) = (q_r, \alpha, L)$

15. $\delta(q_{fb}, b) = (q_r, \beta, L)$

16. $\delta(q_r, a) = (q_r, a, L)$

17. $\delta(q_r, b) = (q_r, b, L)$

18. $\delta(q_r, \alpha) = (q_i, \alpha, R)$

19. $\delta(q_r, \beta) = (q_i, \beta, R)$

20. $\delta(q_i, a) = (q_{aa}, \alpha, R)$

21. $\delta(q_i, b) = (q_{ab}, \beta, R)$

22. $\delta(q_i, \alpha) = (q_A, \alpha, S)$

23. $\delta(q_i, \beta) = (q_A, \beta, S)$

Como tabla:

	a	b	α	β	B
--	---	---	----------	---------	---

q_0	q_{aa}, α, R	q_{ab}, β, R			q_A, B, S
q_{aa}	q_{aa}, a, R	q_{aa}, b, R	q_{fa}, α, L	q_{fa}, β, L	q_{fa}, B, L
q_{ab}	q_{ab}, a, R	q_{ab}, b, R	q_{fb}, α, L	q_{fb}, β, L	q_{fb}, B, L
q_{fa}	q_r, α, L				
q_{fb}		q_r, β, L			
q_r	q_r, a, L	q_r, b, L	q_i, α, R	q_i, β, R	
q_i	q_{aa}, α, R	q_{ab}, β, R	q_A, α, S	q_A, β, S	

Ejercicio 7

Construir una MT que calcule la resta de dos números. Ayuda: se puede considerar la idea de solución propuesta en clase.

$MT = (Q, \Gamma, \delta, q_0, q_A, q_R)$

$Q = \{q_0, q_{ag}, q_f, q_{mbl}, q_{rg}, q_i, q_{mbr}, q_{ff}, q_A, q_R\}$

q_0 : estado inicial

q_{ag} : avanzando a buscar guion

q_f : avanzando hasta el final

q_{mbl} : marcar como blanco e ir a la izquierda

q_{rg} : retrocediendo a buscar guion

q_i : retrocediendo hasta el inicio

q_{mbr} : marcar como blanco e ir a la derecha

q_{ff} : ir hasta el final, añadir un uno y aceptar (resuelve restas donde el primer número es menor al segundo)

alfabeto $\Gamma = \{1, -, B\}$

δ :

- $\delta(q_0, B) = (q_A, B, S)$
- $\delta(q_0, 1) = (q_{ag}, 1, R)$
- $\delta(q_0, -) = (q_A, -, S)$
- $\delta(q_{ag}, 1) = (q_{ag}, 1, R)$
- $\delta(q_{ag}, -) = (q_f, -, R)$
- $\delta(q_{ag}, B) = (q_A, B, S)$
- $\delta(q_f, 1) = (q_f, 1, R)$
- $\delta(q_f, B) = (q_{mbl}, B, L)$
- $\delta(q_{mbl}, 1) = (q_{rg}, B, L)$
- $\delta(q_{mbl}, -) = (q_A, B, S)$

$$11. \delta(q_{rg}, 1) = (q_{rg}, 1, L)$$

$$12. \delta(q_{rg}, -) = (q_i, -, L)$$

$$13. \delta(q_i, 1) = (q_i, 1, L)$$

$$14. \delta(q_i, B) = (q_{mbr}, B, R)$$

$$15. \delta(q_{mbr}, 1) = (q_{ag}, B, R)$$

$$16. \delta(q_{mbr}, -) = (q_{ff}, -, R)$$

$$17. \delta(q_{ff}, 1) = (q_{ff}, 1, R)$$

$$18. \delta(q_{ff}, B) = (q_A, 1, S)$$

Como tabla:

	1	-	B
q_0	$q_{ag}, 1, R$	$q_A, -, S$	q_A, B, S
q_{ag}	$q_{ag}, 1, R$	$q_f, -, R$	q_A, B, S
q_f	$q_f, 1, R$		q_{mbl}, B, L
q_{mbl}	q_{rg}, B, L	q_A, B, S	
q_{rg}	$q_{rg}, 1, L$	$q_i, -, L$	
q_i	$q_i, 1, L$		q_{mbr}, B, R
q_{mbr}	q_{ag}, B, R	$q_{ff}, -, R$	
q_{ff}	$q_{ff}, 1, R$		$q_A, 1, S$

Práctica 2 - La Jerarquía de la Computabilidad

Ejercicio 1

Responder breve y claramente los siguientes incisos:

1. *¿En qué se diferencian los lenguajes recursivos, los lenguajes recursivamente enumerables no recursivos y los lenguajes no recursivamente enumerables?*

Los lenguajes recursivos son aquellos para los que existe alguna MT que los decide, es decir, que lo acepta y siempre para. Corresponden a problemas decidibles

Los lenguajes recursivamente enumerables no recursivos son aquellos para los que existe alguna MT que los acepta, es decir, que, para algunos casos negativos, no paran. Corresponden a problemas computables no decidibles

Los lenguajes no recursivamente enumerables son aquellos para los que no existe ninguna MT que los acepta (a partir de algunos casos positivos, no paran). Corresponden a problemas no computables

2. *Probar que $R \subseteq RE \subseteq \mathcal{U}$. Ayuda: usar las definiciones*

$R \subseteq RE$ si se cumple que, si $L \in R$ entonces $L \in RE$, y eso se cumple puesto que una MT que decida L , también lo va a aceptar (y además no parará nunca), es decir, pertenece a R y a RE

Por otra parte, $RE \subseteq \mathcal{U}$ si se cumple que, si $L \in RE$ entonces $L \in \mathcal{U}$ y esto se cumple porque $\mathcal{U}$ es el conjunto de todos los lenguajes formados por conjuntos de cadenas de Σ^* (el conjunto universal de todas las cadenas de símbolos del alfabeto universal Σ)

3. *Dijimos en clase que el hecho de que si L es recursivo entonces LC también lo es, significa en términos de problemas que si un problema es decidible entonces también lo es el problema contrario. ¿Qué significa en términos de problemas que la intersección de dos lenguajes recursivos es también un lenguaje recursivo?*

Eso significa, en términos de problemas, que si dos problemas son decidibles, también lo será el problema que consiste en resolver los dos problemas a la vez. Por ejemplo, si tenemos P_1 : determinar si un número es par, y P_2 : determinar si un número es primo, y ambos son decidibles, entonces P : determinar si un número es par y primo, también será decidible

4. *Explicar por qué no es correcta la siguiente prueba de que si $L \in RE$, también $L^c \in RE$: dada una MT M que acepta L , entonces la MT M' , igual que M pero con los estados finales permutados, acepta L^c .*

No es correcta la prueba, porque que M acepte L significa que en algunos casos negativos la MT puede looppear, entonces, si se permutan los estados finales, M' looppearía en algunos casos positivos, lo que implicaría que $L \notin RE$

5. *¿Qué lenguajes de la clase CO-RE tienen MT que los aceptan? ¿También los deciden?*

Los que pertenecen a la intersección entre CO-RE y RE (es decir R). Y los

lenguajes de la clase CO-RE que son aceptados también son decididos.

6. *Probar que el lenguaje Σ^* de todas las cadenas y el lenguaje vacío $\emptyset$ son recursivos. Alcanza con plantear la idea general. Ayuda: encontrar MT que los decidan.*

Σ^* es recursivo, ya que existe la MT que para toda entrada $\omega \in \Sigma^*$ la acepta, ya que por definición está en el lenguaje de todas las cadenas, y por ende, siempre se detiene

$\emptyset$ es recursivo, ya que existe la MT que para toda entrada $\omega \notin \emptyset$ la rechaza, ya que por definición no está en el lenguaje vacío, y por ende, siempre se detiene

7. *Probar que todo lenguaje finito es recursivo. Alcanza con plantear la idea general. Ayuda: encontrar una MT que lo decida (pensar cómo definir sus transiciones para cada una de las cadenas del lenguaje).*

Podría haber una MT que tenga una combinación de todas las posibles cadenas de entrada en su función de transición, aceptando o rechazando cada una de ellas, por lo que dicha MT decidiría el lenguaje

Ejercicio 2

Considerando la Propiedad 2 estudiada en clase:

1. *¿Cómo implementaría la copia de la entrada w en la cinta 2 de la MT M ?*

Para cada carácter de la entrada hasta llegar a un blanco, lo escribiría en la cinta 2, por ejemplo, si pudiera haber una 'r', debería estar el estado: $\delta(q_c, (r, B)) = (q_c, (r, R), (r, R))$ (esta entrada de la función de transición se debería replicar para todos los símbolos del alfabeto de la MT) y por último, frenaría haciendo: $\delta(q_c, (B, B)) = (q_{\text{siguiente_estado}}, (B, S), (B, S))$

2. *¿Cómo implementaría el borrado del contenido de la cinta 2 de la MT M ?*

Implementaría un estado donde se vaya hasta la izquierda (o derecha) del todo y luego iría posición por posición poniendo los caracteres en blanco hasta llegar al final de los valores (teniendo en cuenta en ambos casos, posibles blancos que puedan haber debido al procesamiento de la MT M_1)

Ejercicio 3

Probar:

1. *La clase R es cerrada con respecto a la operación de unión. Ayuda: la prueba es similar a la desarrollada para la intersección.*

Para que la clase R sea cerrada con respecto a la operación de unión se debe cumplir que si $L_1 \in R$ y $L_2 \in R$, entonces $L_1 \cup L_2 \in R$. Es decir, si existe la MT M_1 que decide L_1 y existe la MT M_2 que decide L_2 , entonces existe la MT M que decide $L_1 \cup L_2$.

Se construye entonces una MT M que ejecuta secuencialmente las MT M_1 y M_2 y, si M_1 acepta la entrada, M la acepta también, pero si M_1 rechaza la entrada,

pasa a ejecutar M2 con la misma entrada y acepta si M2 acepta y rechaza si M2 lo hace

2. *La clase RE es cerrada con respecto a la operación de intersección. Ayuda: la prueba es similar a la desarrollada para la clase R*

Para que la clase RE sea cerrada con respecto a la operación de intersección se debe cumplir que si $L1 \in RE$ y $L2 \in RE$, entonces $L1 \cap L2 \in RE$. Es decir, si existe la MT M1 que acepta L1 y existe la MT M2 que acepta L2, entonces existe la MT M que acepta $L1 \cap L2$.

Se construye entonces una MT M que ejecuta secuencialmente las MT M1 y M2 y, si M1 rechaza la entrada, M la rechaza también, pero si M1 acepta la entrada, pasa a ejecutar M2 con la misma entrada y acepta si M2 acepta y rechaza si M2 lo hace (en el caso de que se quede loopeando en cualquiera de las dos MTs, M también lo haría)

Ejercicio 4

Sean L1 y L2 dos lenguajes recursivamente enumerables de números naturales codificados en unario (por ejemplo, el número 5 se representa con 11111). Probar que también es recursivamente enumerable el lenguaje $L = \{x \mid x \text{ es un número natural codificado en unario, y existen } y, z, \text{ tales que } y + z = x, \text{ con } y \in L1, z \in L2\}$. Ayuda: la prueba es similar a la vista en clase, de la clausura de la clase RE con respecto a la operación de concatenación

Hay que probar que si existe la MT M1 que acepta L1 y existe la MT M2 que acepta L2, entonces también existe la MT M que acepta L.

Dado un input w con n símbolos, se obtienen diferentes particiones de w: ($\forall x \in$ rango entre 0 y n) toma x dígitos de w para ser ejecutados en M1 y (n - x) dígitos de w para ser ejecutados en M2, pero no se puede realizar una ejecución secuencial, sino que se debe realizar una ejecución “en paralelo” (para evitar loops) donde se ejecuta 1 paso de la primera partición en cada MT (con 0 dígitos de w para la M1 y n dígitos de w para la M), luego 1 paso de la segunda partición en cada MT, y así hasta realizar 1 paso de cada partición y luego se ejecutan 2 pasos de cada partición, y así hasta aceptar (las dos MTs aceptan las dos particiones correspondientes al mismo x) o rechazar (se rechazan todas las particiones) o que quede loopeando en un caso negativo. Es decir, M acepta a L

Ejercicio 5

Dada una MT M1 con alfabeto $\Gamma = \{0, 1\}$:

1. *Construir una MT M2, utilizando la MT M1, que acepte, cualquiera sea su cadena de entrada, si y sólo si la MT M1 acepta al menos una cadena. Ayuda: si M1 acepta al menos una cadena, entonces existe al menos una cadena de símbolos 0 y 1, de tamaño n, tal que M1 la acepta en k pasos. Teniendo en cuenta esto, pensar cómo M2 podría simular M1 considerando todas las cadenas de símbolos 0 y 1 hasta encontrar eventualmente una que M1 acepte (¡cuidándose de los casos en que M1*

entre en loop!).

M2 debería ejecutar un paso de cada posible combinación de los símbolos 0 y 1 y luego 2 pasos de cada combinación y así hasta aceptar, rechazar o quedar loopeando en cada combinación (como se mostró en el ejercicio 4), y en caso de que alguno sea aceptado, aceptar, y sino, rechazar o quedar loopeando

2. *¿Se puede construir además una MT M3, utilizando la MT M1, que acepte, cualquiera sea su cadena de entrada, si y sólo si la MT M1 acepta a lo sumo una cadena? Justificar.*

No, el problema no es computable. Esto es porque en casos donde M1 haya aceptado 0 o 1 cadenas, y el resto de cadenas sean rechazadas o loopeen (no haya más cadenas que M1 acepte), M3 debería aceptar, pero no podrá hacerlo porque M1 quedará loopeando, lo que nos da un problema no computable

Práctica 3 - Indecibilidad

Ejercicio 1

¿Qué es una MT universal?

Una MT universal es una MT que puede ejecutar cualquier otra MT

La misma recibe una MT M codificada en una cadena $\langle M \rangle$ y una cadena de entrada w , la cual ejecutará sobre M

Ejercicio 2

Explicar cómo enumeraría los números naturales pares, los números enteros, los números racionales (o fraccionarios), y las cadenas de Σ^* siendo $\Sigma = \{0, 1\}$

Para enumerar los números naturales pares, lo haría representando cada par P_i como 2^i , donde el 0 (P_0) sería 2^0 , el 2 (P_1) sería 2^1 , etc.

Para enumerar los números enteros lo haría comenzando por el 0, y luego alternando entre positivos y negativos con valor absoluto ascendente, es decir, 0, 1, -1, 2, -2, 3, -3, 4, -4, etc.

Para enumerar los números racionales construiría una tabla donde en ambos ejes estén todos los números naturales (sin el cero), y cada celda representaría la fracción correspondiente a sus extremos: la celda en la posición 1,4 sería $\frac{1}{4}$, la celda en la posición 1,3 sería $\frac{1}{3}$, y luego, en un patrón de zigzag recorrería todos los valores, comenzando por la celda 1,1; 2,1; 1,2; 1,3; 2,2; 3,1; 4,1; 3,2; 2,3; 1,4; etc.

Diagrama de una tabla de fracciones con un camino en zigzag verde:

	1	2	3	4	5	6	...
1	1/1	2/1	3/1	4/1	5/1	6/1	...
2	1/2	2/2	3/2	4/2	5/2	6/2	...
3	1/3	2/3	3/3	4/3	5/3	6/3	...
4	1/4	2/4	3/4	4/4	5/4	6/4	...
5	1/5	2/5	3/5	4/5	5/5	6/5	...
6	1/6	2/6	3/6	4/6	5/6	6/6	...
...	...	...	...	...	...	...	...

El camino en zigzag verde comienza en (1,1) y recorre la tabla en la siguiente secuencia: (1,1) → (2,1) → (2,2) → (1,2) → (1,3) → (2,3) → (3,3) → (4,3) → (4,2) → (5,2) → (5,1) → (6,1) → (6,2) → (6,3) → (6,4) → (5,4) → (4,4) → (3,4) → (2,4) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5) → (6,5) → (6,6) → (5,6) → (4,6) → (3,6) → (2,6) → (1,6) → (1,7) → (2,7) → (3,7) → (4,7) → (5,7) → (6,7) → (7,7) → (7,6) → (7,5) → (7,4) → (7,3) → (7,2) → (7,1) → (6,1) → (5,1) → (4,1) → (3,1) → (2,1) → (1,1).

Las cadenas de Σ^* siendo $\Sigma = \{0, 1\}$ se podrían enumerar según el orden canónico, donde se enumeran primero todas las cadenas de longitud cero, luego las de longitud 1, y así sucesivamente, y para las que son de la misma longitud, se las enumera alfabéticamente (el 0 va antes que el 1), por lo que quedaría de la siguiente forma:

0. λ
1. 0, 1
2. 00, 01, 10, 11
3. 000, 001, 010, 011, 100, 101, 110, 111

4. 0000, 0001, 0010, 0011, 0100, 0101, 0110, etc.

Ejercicio 3

Dar la idea general de cómo sería una MT que, teniendo como cadena de entrada un número natural i , genera la i -ésima fórmula booleana satisfactible según el orden canónico

Comentario: asumir que existen una MT M_1 que determina si una cadena es una fórmula booleana, y una MT M_2 que determina si una fórmula booleana es satisfactible

Se deberían seguir los siguientes pasos:

1. $x = 0$ (se inicializa el contador)
2. Se obtiene la siguiente cadena c del orden canónico
3. Se pasa c por la M_1 :
 - a. Si c es una fórmula booleana avanza al paso 4
 - b. Si c no es una fórmula booleana vuelve al paso 2 (para continuar con la siguiente cadena del orden canónico)
4. Se pasa c por la M_2 :
 - a. Si c es una fórmula booleana satisfactible avanza al paso 5
 - b. Si c no es una fórmula booleana satisfactible vuelve al paso 2 (para continuar con la siguiente cadena del orden canónico)
5. c es la fórmula booleana satisfactible número x :
 - a. Si $x=i$, se retorna c
 - b. Si no, se incrementa x en 1 y se vuelve al paso 2 (para continuar con la siguiente cadena del orden canónico)

Ejercicio 4

Sea M_1 una MT que genera en su cinta de salida todas las cadenas de un lenguaje L . Dar la idea general de cómo sería una MT M_2 que, usando M_1 , acepte una cadena w si y sólo si $w \in L$

M_2 debería ejecutar M_1 y para cada cadena c que encuentre en su cinta de salida, debería compararla con w . Si $c=w$, acepta, sino, sigue revisando hasta que $c=w$ o que M_1 termine de generar cadenas (si esto último pasa, M_2 debería rechazar)

Ejercicio 5

El lenguaje $L_U = \{ \langle M \rangle, w \mid M \text{ acepta } w \}$ se conoce como lenguaje universal, y representa el problema general de aceptación. Probar que $L_U \in RE$. Ayuda: construir una MT que acepte L_U

Una MT M_{LU} que acepte a L_U sería una donde se ejecute M con cadena de entrada w . Si M acepta a w , entonces M_{LU} acepta a $\langle M \rangle, w$, si M rechaza a w , entonces M_{LU}

rechaza a $\langle M \rangle, w$ y si M loopea M_{LU} también lo hace. De esta manera, M_{LU} acepta a L_U ya que puede no detenerse para casos negativos, por lo que, $L_U \in RE$

Ejercicio 6

Una función $f : A \rightarrow B$ es total computable si y sólo si existe una MT M_f que la computa para todo elemento $a \in A$

Sea la función $f_{01} : \Sigma^* \rightarrow \{0, 1\}$ tal que:

- $f_{01}(v) = 1$, si $v = \langle M \rangle, w$ y M para a partir de w
- $f_{01}(v) = 0$, si $v = \langle M \rangle, w$ y M no para a partir de w o bien $v \neq \langle M \rangle, w$

Probar que f_{01} no es total computable

Ayuda: ¿con qué problema se relaciona dicha función?

f_{01} no será total computable si para al menos un elemento $v \in \Sigma^*$, M_f no lo computa, es decir, loopea

Esto sucederá en algunos de los casos donde $f_{01}(v)$ debería ser igual a 0, ya que, si bien M no parará a partir de w , M_f podría entrar en un loop y seguir ejecutándose, implicando que no podrá computar dicho elemento v

Ejercicio 7

Responder breve y claramente cada uno de los siguientes incisos (en todos los casos, las MT mencionadas tienen una sola cinta):

- Probar que se puede decidir si una MT M , a partir de la cadena vacía λ , escribe alguna vez un símbolo no blanco. Ayuda: ¿Cuántos pasos puede hacer M antes de entrar en un loop?

Habría que construir una MT M' que reciba $\langle M \rangle$ y ejecute M a partir de λ y que, si M en algún momento escribe un símbolo no blanco, M' acepte y si eso nunca sucede, M' debería rechazar

Esta es una versión reducida del Halting Problem, donde N es la cantidad de configuraciones distintas y se calcula como $N = K * N_1 * N_2^K$, y donde la MT M debe tener 1 cinta, debe ocupar un espacio acotado de celdas, y tiene N_1 estados, N_2 símbolos y recorre a lo sumo K celdas para toda entrada w

En este caso, como N_2 es 1 (ya que al escribir un símbolo distinto de blanco M' acepta), entonces $N = K * N_1$

Por último, como todas las posiciones de las cintas están en blanco (y al modificar eso, M' aceptaría), entonces K también es 1 (ya que todas las posiciones son indiferenciables entre sí), por lo que $N = N_1$ que es la cantidad de estados de M , la cual, por definición, es finita

Entonces, si M ejecuta N pasos sin escribir un símbolo no blanco, podemos estar seguros que está en un loop y M' rechaza, por otra parte, si

escribe un símbolo no blanco antes, M' acepta

- b. *Probar que se puede decidir si una MT M que sólo se mueve a la derecha, a partir de una cadena w , para, Ayuda: ¿Cuántos pasos puede hacer M antes de entrar en un loop?*

Habría que construir una MT M' que reciba $\langle M \rangle, w$ y ejecute M a partir de w y que, si M en algún momento para, M' acepte y si eso nunca sucede, M' debería rechazar

Esta es una versión reducida del Halting Problem, donde N es la cantidad de configuraciones distintas y se calcula como $N = K * N_1 * N_2^K$, y donde la MT M debe tener 1 cinta, debe ocupar un espacio acotado de celdas, y tiene N_1 estados, N_2 símbolos y recorre a lo sumo K celdas para toda entrada w

En este caso K es virtualmente finito, ya que como solo se puede mover a la derecha, una vez que llega al fin de w (que por definición es finita), sólo habrá blancos

De esta manera, como N_1 y N_2 son por definición finitos, N también lo será

Entonces, si M ejecuta N pasos sin parar, podemos estar seguros que está en un loop, por lo que nunca parará, y M' rechaza, por otra parte, si M para antes de los N pasos, M' acepta

- c. *Probar que se puede decidir si dada una MT M , existe una cadena w a partir de la cual M para en a lo sumo 10 pasos. Ayuda: ¿Hasta qué tamaño de cadenas hay que chequear?*

Habría que construir una MT M' que reciba $\langle M \rangle$ y ejecute M a partir de cadenas de longitud creciente y que, si M acepta o rechaza en a lo sumo 10 pasos, M' acepte y si eso nunca sucede, M' debería rechazar

En este caso, como el límite es de 10 pasos, solo se deberían chequear las cadenas de hasta 10 símbolos en el orden canónico, esto es así porque, si para alguna de las cadenas de 10 o menos M se detiene, entonces M' acepta, y si eso no sucede, sabemos que para ninguna de las cadenas de 11 o más símbolos M se detendrá, porque para cada una de ellas, los primeros 10 símbolos pertenecen a alguna de las cadenas de 10 símbolos ya chequeadas, y como M tiene un límite de 10 pasos (antes de que M' la descarte), nunca podrá verificar el símbolo 11 de la cadena, haciendo que si, para las cadenas ya vistas no paró, tampoco lo haga para las de longitud superior a 10

De esta manera, chequeando las cadenas de longitud 10 o menos por hasta 10 pasos de M , M' decidirá siempre, ya que ambas cantidades (cantidad de cadenas a chequear y cantidad de pasos de M) son finitas

- d. *¿Se puede decidir si dada una MT M , existe una cadena w de a lo sumo 10 símbolos a partir de la cual M para? Justificar la respuesta*

Si se puede decidir significa que existe una MT M' que para la entrada $\langle M \rangle$ decida si existe una cadena w de a lo sumo 10 símbolos a partir de la cual M se detiene

Habría que construir M' que reciba $\langle M \rangle$ y ejecute M a partir de cadenas de longitud creciente de forma "paralela", y que si M se detiene para alguna de ellas, M' acepte y si no lo hace para ninguna, M' rechace

Para ejecutar de forma "paralela" se hace:

1. $i = 1$
2. Se ejecuta i pasos para cada cadena de hasta 10 símbolos
3. Si para alguna de las cadenas M se detiene, M' acepta
4. Si no, incrementar i en 1 y volver al paso 2

El problema con esta solución es que si para todas las cadenas de a lo sumo 10 símbolos, M no se detiene, M' tampoco lo hará

Para que M' decida, entonces, habría que probar que se puede decidir si M para o no, eso es el Halting Problem, y como en este caso M no tiene un espacio acotado de celdas, no se puede decidir si M para o no, por lo que M' no decide, sino que acepta y en algunos casos negativos loopea ($L(M') \in (RE - R)$)

Práctica 4 - Las reducciones

Ejercicio 1

Considerando la reducción de HP a L_U descrita en clase, responder:

- a. Explicar por qué la función identidad, es decir la función que a toda cadena le asigna la misma cadena, no es una reducción de HP a L_U

La función identidad realiza una reducción de L a L , es decir, si se aplica la función identidad a HP, se obtiene HP. Esta es la propiedad de reflexividad, la cual dice que se cumple que $L \leq L$

En el caso de HP a L_U , para los casos donde HP acepte a partir de w porque M_1 paró por q_A , L_U (luego de aplicada la función identidad) aceptaría, pero para los casos donde HP acepte a partir de w porque M_1 paró por q_R , L_U no aceptaría (ya que se hizo q_R). Entonces, existe al menos ese caso donde $(\langle M \rangle, w) \in HP$, pero $f(\langle M \rangle, w) \notin L_U$, por lo que la función identidad no es una reducción de HP a L_U

- b. Explicar por qué las MT M_2 generadas en los pares de salida $(\langle M_2 \rangle, w)$, o bien paran aceptando, o bien loopean

Esto es porque, si M_1 acepta, se puede dar porque se hizo q_A o q_R . Y luego de aplicada la reducción, los q_R , se reemplazan por q_A , por lo que siempre que pare M_1 a partir de $(\langle M \rangle, w)$, M_2 aceptará a partir de $f(\langle M \rangle, w)$. Por otra parte, cuando M_1 loopee, M_2 también lo hará, ya que la reducción utilizada sólo afecta a los q_A y q_R

- c. Explicar por qué la función utilizada para reducir HP a L_U también sirve para reducir HP^C a L_U^C

Por la definición de reducción, ya que la función utilizada para reducir HP a L_U dice que $(\langle M \rangle, w) \in HP$ si y sólo si $f(\langle M \rangle, w) \in L_U$ o lo que es lo mismo, $(\langle M \rangle, w) \notin HP$ si y sólo si $f(\langle M \rangle, w) \notin L_U$. Entonces, $(\langle M \rangle, w) \in HP^C \leftrightarrow (\langle M \rangle, w) \notin HP \leftrightarrow f(\langle M \rangle, w) \notin L_U \leftrightarrow f(\langle M \rangle, w) \in L_U^C$. Es decir, con la misma función utilizada para reducir HP a L_U se puede reducir HP^C a L_U^C (esto se cumple para todos los lenguajes: si $L_1 \leq L_2$, entonces $L_1^C \leq L_2^C$, con la misma reducción)

- d. Explicar por qué la función utilizada para reducir HP a L_U no sirve para reducir L_U a HP

Porque, si se hace eso, se cambiarían los q_R por q_A , es decir, los estados rechazados por M_2 (donde w no sea aceptada por M_2) se detendrían, por lo que habría $f(\langle M \rangle, w) \notin L_U$ pero donde $(\langle M \rangle, w) \in HP$ (ya que dicho w para y por ende es aceptado por M_1) (se llega al mismo resultado si se aplica la función identidad, ya que tanto q_R , como q_A , se detienen y son aceptados por M_1)

- e. Explicar por qué la siguiente MT M_f no computa una reducción de HP a L_U : dada una cadena válida $(\langle M \rangle, w)$, M_f ejecuta M sobre w , si M acepta

entonces genera la salida $\langle M \rangle, w$, y si M rechaza entonces genera la cadena 1

Dada una cadena $\langle M \rangle, w$ a partir de la cual la MT M para por q_R , M_1 ($L(M_1) = HP$) acepta dicha cadena, pero si se la pasa como entrada a la MT M_f , la misma producirá un 1 (ya que M para por q_R), el cual será rechazado por M_2 ($L(M_2) = L_U$) por no ser una cadena válida. Entonces, existe al menos ese caso donde $\langle M \rangle, w \in HP$, pero $f(\langle M \rangle, w) \notin L_U$, por lo que la MT M_f no computa una reducción de HP a L_U

Ejercicio 2

Sabiendo que $L_U \in RE$ y $L_U^C \in CO-RE$:

- a. Probamos en clase que existe una reducción de L_U a L_{Σ^*} . En base a esto, ¿qué se puede afirmar con respecto a la ubicación de L_{Σ^*} en la jerarquía de la computabilidad?

Si $L_U \leq L_{\Sigma^*}$, entonces $L_U^C \leq L_{\Sigma^*}^C$

Si $L_U^C \in CO-RE$, entonces $L_U^C \notin RE$

Si $L_U^C \leq L_{\Sigma^*}^C$, y $L_U^C \notin RE$, entonces $L_{\Sigma^*}^C \notin RE$

Sabemos que L_{Σ^*} , es tan o más difícil que L_U , es decir, es al menos, RE-COMPLETO, y también sabemos que $L_{\Sigma^*}^C \notin RE$

Y de hecho, en la teoría se menciona que $L_U^C \leq L_{\Sigma^*}$, entonces.

Si $L_U^C \leq L_{\Sigma^*}$, y $L_U^C \notin RE$ entonces $L_{\Sigma^*} \notin RE$

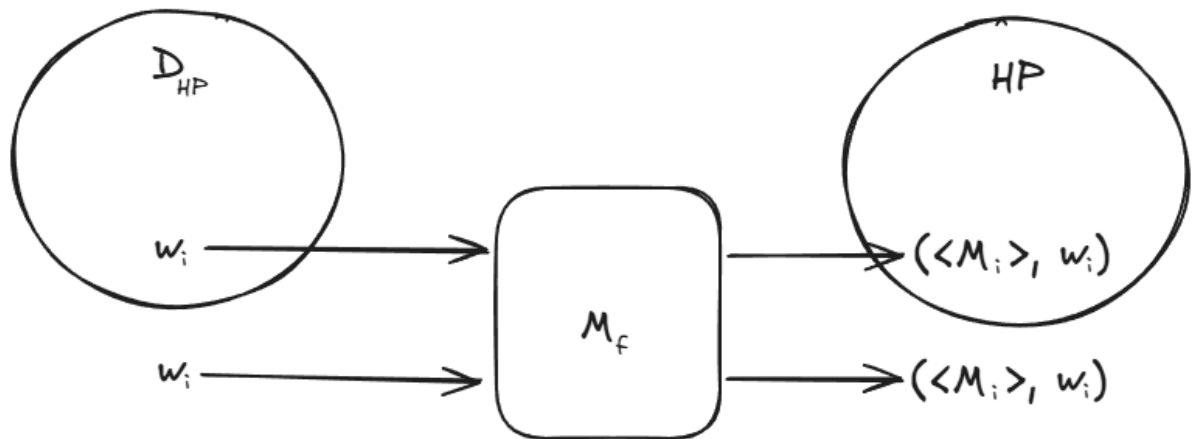
Si $L_{\Sigma^*} \notin RE$ y $L_{\Sigma^*}^C \notin RE$, entonces, L_{Σ^*} y $L_{\Sigma^*}^C$ pertenecen ambos a $\mathcal{Q} - (RE \cup CO-RE)$

- b. Se prueba que existe una reducción de L_U^C a $L_{\emptyset}$. En base a esto, ¿qué se puede afirmar con respecto a la ubicación de $L_{\emptyset}$ en la jerarquía de la computabilidad?

Si $L_U^C \leq L_{\emptyset}$, y $L_U^C \notin RE$, entonces $L_{\emptyset} \notin RE$

Ejercicio 3

Sea el lenguaje $D_{HP} = \{w_i \mid M_i \text{ para a partir de } w_i\}$ (considerar el orden canónico). Encontrar una reducción de D_{HP} a HP . Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud



Reducción (gráfico): $\langle M_i \rangle$ es la MT del orden canónico, tal que M_i se detiene a partir de w_i

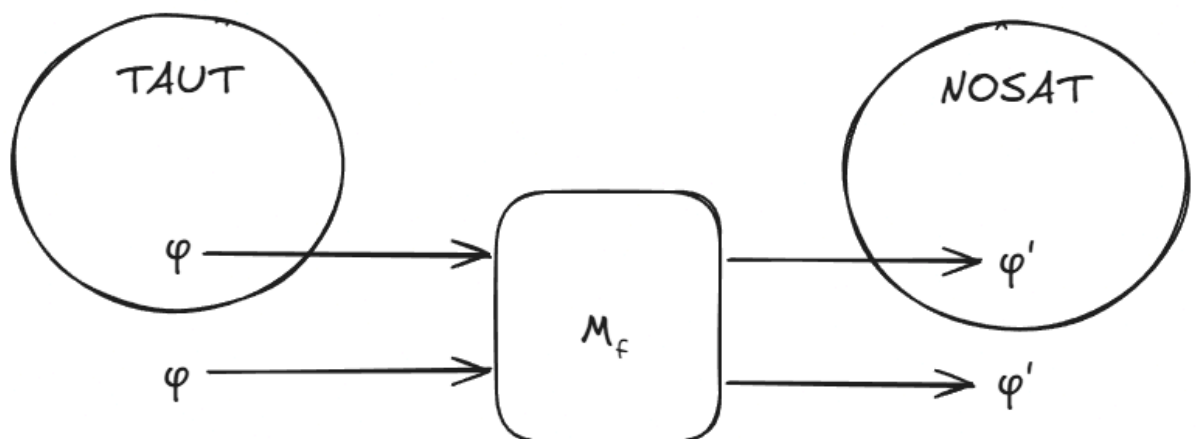
Computabilidad: M_f obtiene i , en base a w_i , generando todas las cadenas en orden canónico hasta encontrar w_i , y luego obtiene M_i , en base a i , generando todas las cadenas en orden canónico hasta encontrar la MT válida número i (como se vio en la clase 3) (por lo que $f(w_i)$ es total computable, al realizarse una búsqueda sobre el orden canónico)

Correctitud:

- $w_i \in D_{HP} \rightarrow M_i$ se detiene a partir de $w_i \rightarrow (\langle M_i \rangle, w_i) \in HP$
- $w_i \notin D_{HP} \rightarrow M_i$ no se detiene a partir de $w_i \rightarrow (\langle M_i \rangle, w_i) \notin HP$

Ejercicio 4

Sean *TAUT* y *NOSAT* los lenguajes de las fórmulas booleanas sin cuantificadores tautológicas (satisfactibles por todas las asignaciones de valores de verdad) e insatisfactibles (insatisfactibles por todas las asignaciones de valores de verdad). Encontrar una reducción de *TAUT* a *NOSAT*. Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud



Reducción (gráfico): ϕ' es una fórmula booleana, que es la negación de ϕ

Computabilidad: M_f niega ϕ (por lo que es total computable)

Correctitud:

- $\varphi \in \text{TAUT} \rightarrow \varphi$ es una fórmula booleana satisfactible por todas las asignaciones de valores de verdad $\rightarrow \varphi'$ es una fórmula booleana insatisfactible por todas las asignaciones de valores de verdad (por la construcción de M_f) $\rightarrow \varphi' \in \text{NOSAT}$
- $\varphi \notin \text{TAUT} \rightarrow \varphi$ es una fórmula booleana insatisfactible para al menos una asignación de valores de verdad $\rightarrow \varphi'$ es una fórmula booleana satisfactible para al menos una asignación de valores de verdad (por la construcción de M_f) $\rightarrow \varphi' \notin \text{NOSAT}$

Ejercicio 5

Se prueba que existe una reducción de L_U^C a L_{Σ^} (y así, como $L_U^C \notin RE$, entonces se cumple que $L_{\Sigma^*} \notin RE$). La reducción es la siguiente. Para toda w : $f(\langle M_1 \rangle, w) = \langle M_2 \rangle$, tal que M_2 , a partir de su entrada v , ejecuta $|v|$ pasos de M_1 a partir de w , y acepta si y sólo si M_1 no acepta. Probar que la función definida es efectivamente una reducción de L_U^C a L_{Σ^*} . Comentario: hay que probar su total computabilidad y correctitud*

Computabilidad: M_f construye una MT M_2 que ejecuta $|v|$ pasos (sea v su entrada) de M_1 a partir de w y acepte si y sólo si M_1 no acepta en esos $|v|$ pasos (porque rechaza, loopea o no llega a un q_R/q_A). Por lo tanto $f(\langle M_1 \rangle, w)$ es total computable, puesto que M_f decide, ya que ejecuta $|v|$ pasos de M_1 y, por definición, una entrada de una MT (como lo es v) es finita

Correctitud:

- $(\langle M_1 \rangle, w) \in L_U^C \rightarrow M_1$ rechaza (o loopea) a partir de $w \rightarrow M_2$ acepta (porque M_1 rechaza w , o no llega a aceptar en $|v|$ pasos (porque loopea o simplemente no llega a un q_R)) (por la construcción de M_f) $\rightarrow \langle M_2 \rangle \in L_{\Sigma^*}$
- $(\langle M_1 \rangle, w) \notin L_U^C$ y M_1 es válida $\rightarrow M_1$ acepta a partir de $w \rightarrow M_2$ rechaza (porque M_1 acepta w para al menos 1 caso donde $|v|$ llegue a ser lo suficientemente alto como para que se ejecuten los suficientes pasos para llegar al q_A) (por la construcción de M_f) $\rightarrow \langle M_2 \rangle \notin L_{\Sigma^*}$
- $(\langle M_1 \rangle, w) \notin L_U^C$ y M_1 no es válida $\rightarrow M_1$ no es válida $\rightarrow M_2$ no es válida $\rightarrow \langle M_2 \rangle \notin L_{\Sigma^*}$

Práctica 5 - Tiempo polinomial y no polinomial

Ejercicio 1

Responder breve y claramente los siguientes incisos:

- a. *¿Por qué la complejidad temporal sólo trata los lenguajes recursivos?*

La complejidad temporal sólo trata a los lenguajes recursivos porque sólo en ellos, al ser decidibles (no looppear), se puede realizar un cálculo correcto de su complejidad temporal, ya que no tienen casos donde nunca terminen

- b. *Probar que $n^3 = O(2^n)$. Ayuda: hay que encontrar un número natural n_0 y una constante $c > 0$ tales que $n^3 \leq c \cdot 2^n$ para todo $n \geq n_0$*

Con $c = 1$, $n_0 = 10$, n^3 siempre será menor que $c \cdot 2^n$, ya que 2^n crece más rápido que n^3

- c. *Probar que si $T_1(n) = O(T_2(n))$, entonces $TIME(T_1(n)) \subseteq TIME(T_2(n))$. Ayuda: hay que probar que si un lenguaje L está en $TIME(T_1(n))$, también está en $TIME(T_2(n))$. Recurrir directamente a la definición de orden O de una función T y de clase $TIME(T(n))$*

Esto es porque $L \in TIME(T_1(n))$ si y sólo si existe una MT M que decide a L en tiempo $O(T_1(n))$ (para cada entrada w , con $|w| = n$, hace a lo sumo $T_1(n)$). Y, si $T_1(n)$ es de orden $T_2(n)$, entonces $T_1(n) \leq c \cdot T_2(n)$ ($c > 0$), por lo que M decide a L en tiempo $O(c \cdot T_2(n))$, es decir, $L \in TIME(T_2(n))$

- d. *¿Cuándo un lenguaje pertenece a P , a NP y a EXP ? ¿Por qué si un lenguaje pertenece a P también pertenece a NP y a EXP ?*

Un lenguaje pertenece a P si existe una MT que lo acepta en tiempo $poly(n)$

Un lenguaje pertenece a NP si existe una MT que lo verifica (dado un certificado C) en tiempo $poly(n)$

Un lenguaje pertenece a EXP si existe una MT que lo acepta en tiempo $exp(n)$

Si un lenguaje pertenece a P , también lo hará a NP y a EXP porque si existe una MT que lo decide en tiempo $poly(n)$, también lo podrá verificar en $poly(n)$ (pertenece a NP) y $poly(n) = O(exp(n))$ (pertenece a EXP)

- e. *¿Qué formula la Tesis Fuerte de Church-Turing?*

Dice que todo si el lenguaje L es decidible en tiempo $poly(n)$ por modelo computacional físicamente realizable, entonces existe una MT que lo decide en tiempo $poly(n)$

- f. *¿Por qué es indistinta la cantidad de cintas de las MT que utilizamos para analizar los lenguajes, en el marco de la jerarquía temporal que definimos?*

Porque al variar la cantidad de cintas, los tiempos $poly(n)$ siguen

siéndolo, por lo que, si en una MT de k cintas resuelvo un problema en tiempo $\text{poly}(n)$, entonces existe una MT de 1 cinta que lo resuelve en tiempo $\text{poly}(n)$

g. ¿Qué codificación de cadenas se descarta en la complejidad temporal?

La unaria

h. ¿Por qué si un lenguaje L pertenece a P , también su complemento L^c pertenece a P ? Ayuda: hay que probar que si existe una MT M que decide L en tiempo $\text{poly}(n)$, también existe una MT M' que decide L^c en tiempo $\text{poly}(n)$

Esto es así porque si la MT M decide a L en tiempo $\text{poly}(n)$, entonces se podría construir una MT M' que decida a L^c en tiempo $\text{poly}(n)$ haciendo que la misma ejecute a M y, si M rechaza, entonces M' aceptará, y si M acepta, M' rechazará. Dado que L pertenece a P , su ejecución es en tiempo $\text{poly}(n)$, por lo que también lo es la ejecución de M' , y por ende, L^c también está en P

i. Sea L un lenguaje de NP. Explicar por qué los certificados de L miden un tamaño polinomial con respecto al tamaño de las cadenas de entrada.

Porque si L está en NP, significa que se puede verificar a L , a partir de un certificado x , en tiempo $\text{poly}(n)$, por lo que si el certificado de L midiera un tamaño exponencial con respecto al tamaño de las cadenas de entrada, tan solo el tiempo de leer el certificado haría que su verificación no sea en tiempo $\text{poly}(n)$, por lo que L no estaría en NP

Ejercicio 2

Sea el lenguaje $\text{SMALL-SAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana sin cuantificadores en la forma normal conjuntiva (o FNC), y existe una asignación de valores de verdad que la satisface en la que hay a lo sumo 3 variables con valor de verdad verdadero}\}$. Probar que $\text{SMALL-SAT} \in P$

Comentario: las fórmulas φ en la forma FNC son conjunciones de disyunciones de variables o variables negadas; p.ej. $(x_1 \vee x_2) \wedge x_4 \wedge (\neg x_3 \vee x_5 \vee x_6)$

Ayuda: una MT que decide SMALL-SAT debe contemplar asignaciones con cero, uno, dos y hasta tres valores de verdad verdadero

Para probar que $\text{SMALL-SAT} \in P$, habría que construir una MT M que reciba φ y decida a SMALL-SAT en tiempo $\text{poly}(\varphi)$

Para decidir a SMALL-SAT se deben verificar las combinaciones de valores de verdad donde haya cero, uno, dos y hasta tres valores de verdad verdaderos

Para cada uno de esos casos basta con verificar $C(|\varphi|, x)$ combinaciones de valores de verdad, donde x es 0, 1, 2 o 3 (cantidad de valores de verdad verdaderos) y C es la cantidad de combinaciones de $|\varphi|$ obtenidos de a x valores

Como hacer $C(|\varphi|, x)$ implica hacer $\frac{|\varphi|!}{(x!(|\varphi|-x)!)}$:

- Cuando $x = 0$, $C(|\varphi|, 0)$ vale 1, lo cual se hace en $O(1)$
- Cuando $x = 1$, $C(|\varphi|, 1)$ vale $|\varphi|$, lo cual se hace en $O(|\varphi|)$

- Cuando $x = 2$, $C(|\phi|, 2)$ vale $(|\phi| * (|\phi| - 1))/2$, lo cual se hace en $O(|\phi|^2)$
- Cuando $x = 3$, $C(|\phi|, 2)$ vale $(|\phi| * (|\phi| - 1) * (|\phi| - 2))/6$, lo cual se hace en $O(|\phi|^3)$

Entonces, dado que cada cantidad de combinaciones por valor de x es de orden de $O(|\phi|^3)$, lo cual es de orden $\text{poly}(|\phi|)$, y que verificar cada una de esas combinaciones se hace en $\text{poly}(|\phi|)$, entonces, SMALL-SAT está en P

Ejercicio 3

Dados los dos lenguajes siguientes, (1) justificar por qué no estarían en P, (2) probar que están en NP, (3) justificar por qué sus complementos no estarían en NP:

- El problema del conjunto dominante de un grafo consiste en determinar si un grafo no dirigido tiene un conjunto dominante de vértices. Un subconjunto D de vértices de un grafo G es un conjunto dominante de G , si y sólo si todo vértice de G fuera de D es adyacente a algún vértice de D . El lenguaje que representa el problema es $\text{DOM-SET} = \{(G, K) \mid G \text{ es un grafo no dirigido y tiene un conjunto dominante de } K \text{ vértices}\}$

- DOM-SET no estaría en P porque debo realizar combinaciones de K elementos del grafo G , por lo que realizaría $C(|V|, K)$ instancias donde para cada una verifico que los $|V| - K$ nodos restantes de G estén conectados a algún nodo de los elegidos, $C(|V|, K)$ es de orden $|V|^K$, y como K depende del tamaño de la entrada, $O(|V|^K)$ es $\exp(|(G, K)|)$, por lo que DOM-SET no estaría en P
- DOM-SET está en NP si se verifica que un par (G, K) está en DOM-SET a partir de un certificado x , donde el certificado x es una combinación de K vértices de G , en tiempo polinomial. Para verificar si la combinación presente en x es un conjunto dominante, habría que ir por cada vértice que no esté en x , y verificar que exista una arista de dicho vértice a alguno de los vértices de x , lo cual se realiza en tiempo $O(|V| * |E|)$ (para cada vértice miro todas las aristas), lo cual es de tiempo $\text{poly}(|(G, K)|)$, por lo que DOM-SET es verificado en tiempo polinomial con respecto a su entrada, y por ende, pertenece a NP
- DOM-SET^c no estaría en NP ya que para eso se debería verificar que un par (G, K) está en DOM-SET^c a partir de un certificado x , donde el certificado x es una combinación de K vértices de G , en tiempo polinomial. Pero, para eso, el certificado x debería ser de un tamaño exponencial con respecto a la entrada, ya que debería incluir todas las posibles combinaciones de K vértices de G que no sean conjuntos dominantes, y como vimos en el paso 1), eso es de orden $\exp(|(G, K)|)$, por lo que DOM-SET^c no estaría en NP (ya que si su certificado es de tamaño exponencial con respecto a su entrada, tan solo leer el certificado hará que la verificación sea del orden $\exp(|(G, K)|)$)

b. El problema de los grafos isomorfos consiste en determinar si dos grafos son isomorfos. Dos grafos son isomorfos si son idénticos salvo por la denominación de sus arcos. P.ej., dado el grafo $G_1 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 3), (3, 4), (4, 1)\})$, el grafo $G_2 = (\{1, 2, 3, 4\}, \{(1, 2), (2, 4), (4, 3), (3, 1)\})$ es isomorfo a G_1 . El lenguaje que representa el problema de los grafos isomorfos es $ISO = \{(G_1, G_2) \mid G_1 \text{ y } G_2 \text{ son grafos isomorfos}\}$

1) ISO no estaría en P porque para saber si dos grafos son isomorfos, se puede renombrar cada vértice de G_2 y crear un grafo G_2' , donde sus aristas son iguales que las de G_1 , G_1 y G_2 son grafos isomorfos. Hay $|V_2|!$ de permutaciones posibles para los vértices de G_2 , por lo que sin importar el tiempo que tome el chequeo de igualdad de aristas entre G_1 y G_2' , como se hacen una cantidad de permutaciones de $|V_2|! = |V_2| * (|V_2| - 1) * (|V_2| - 2) * \dots = O(|V_2|^{|V_2|}) = \exp(|G_1, G_2|)$, el tiempo que toma decidir ISO es $\exp(|G_1, G_2|)$, por lo que no estaría en P

2) ISO está en NP si la verificación de una entrada (G_1, G_2) , en base a un certificado x (que contiene una permutación de los vértices de G_2), se realiza en tiempo polinomial. Y esto es así ya que dicha verificación implica:

- Verificar que los vértices del certificado estén en G_2 (y no sobre ni falte ninguno) = $O(|V_2|)$
- Crear G_2' que utilice como base las aristas de G_2 , pero con la permutación de vértices presentes en el $x = O(|E_2| * |V_2|)$
- Verificar que las aristas de G_2' son iguales a las de G_1 (están todas y no sobra ninguna) = $O(|V_1| * |V_2|)$

Entonces, la verificación tarda: $O(|V_2|) + O(|E_2| * |V_2|) + O(|V_1| * |V_2|) = \text{poly}(|G_1, G_2|)$, por lo que ISO pertenece a NP

3) ISO^C no estaría en NP porque su certificado debería incluir todas las permutaciones de vértices de G_2 para verificar que G_1 y G_2 no son isomorfos. Y como vimos en el inciso 1), esa cantidad de permutaciones es de orden $\exp(|G_1, G_2|)$, por lo que tan solo leer el certificado implicaría tardar un tiempo exponencial con respecto a la entrada de ISO^C , por lo que no estaría en NP

Ejercicio 4

Se prueba que $NP \subseteq EXP$. La prueba es la siguiente. Si $L \in NP$, entonces existe una MT M que, para toda cadena de entrada w , verifica en tiempo $\text{poly}(|w|)$ si $w \in L$, con la ayuda de un certificado x tal que $|x| \leq p(|w|)$ - p es un polinomio -, y de esta manera, se puede construir una MT M' que decida en tiempo $\exp(|w|)$ si $w \in L$, sin usar ninguna cadena adicional: M' simplemente barre todos y cada uno de los certificados posibles x de w . Se pide explicar por qué M' efectivamente tarda tiempo $\exp(|w|)$. Ayuda: como $|x| \leq p(|w|)$ y los símbolos de x pertenecen a un alfabeto de k símbolos, ¿cuántos certificados x de tamaño $p(|w|)$ tiene a lo sumo una cadena w ?

Una cadena w tendrá a lo sumo $k^{p(|w|)}$ certificados ya que $|x|$ es menor o igual a $p(|w|)$, y para cada símbolo de x se puede elegir uno de un alfabeto de k símbolos

Por otro lado, como sabemos que $p(|w|)$ es $\text{poly}(|w|)$, entonces $k^{\text{poly}(|w|)}$ es $\exp(|w|)$

Práctica 6

Ejercicio 1

Responder breve y claramente:

- a. *Dar un esquema de prueba de la transitividad de las reducciones polinomiales. Ayuda: en clase hicimos lo propio con las reducciones generales*

Las reducciones polinomiales son transitivas si $L_1 \leq_p L_2$ y $L_2 \leq_p L_3$ entonces $L_1 \leq_p L_3$

Y eso sucede ya que:

- Puedo construir una MT M que decida a L_1 y que reciba w , y haga una reducción a L_2 lo cual existe por la hipótesis, y luego $f(w)$, será reducido de L_2 a L_3 lo cual también existe por la hipótesis
- También sabemos que si $w \in L_1$, entonces $f(w) \in L_2$ y $g(f(w)) \in L_3$ (por la definición de reducción), entonces sabemos que la MT M reduce de L_1 a L_3
- Además, sabemos que dicha reducción es polinómica porque:
 - La reducción de L_1 a L_2 lo es (por la hipótesis), por lo que $f(w) = \text{poly}(|w|)$
 - La reducción de L_2 a L_3 también es polinómica (por la hipótesis), por lo que $g(f(w)) = \text{poly}(|f(w)|) = \text{poly}(\text{poly}(|w|))$. Ya que $|f(w)|$ será menor o igual a $\text{poly}(|w|)$ dado que la MT que reduce de L_1 a L_2 lo hace en tiempo polinómico y su salida ($f(w)$) no podrá ser mayor a $\text{poly}(|w|)$ (no podrá ser mayor a la cantidad de pasos que hace ($\text{poly}(|w|)$))
- Entonces la reducción de L_1 a L_3 se hará en tiempo $\text{poly}(|w|) + \text{poly}(\text{poly}(|w|)) = \text{poly}(|w|)$, por lo que $L_1 \leq_p L_3$

- b. *¿Cuándo un lenguaje es NP-difícil y cuándo es NP-completo?*

CONTINUAR CHEQUEANDO CON GPT Y FRANCO A PARTIR DE ACA

Un lenguaje L es NP-completo si:

- $L \in \text{NP}$
- L es NP-difícil, es decir, para todo $L_i \in \text{NP}$ se cumple que $L_i \leq_p L$

- c. *¿Por qué si $P \neq \text{NP}$, un lenguaje NP-completo no pertenece a P ?*

Porque un lenguaje que pertenece a NPC, es un lenguaje para el cual existe una reducción polinomial de todos los lenguajes de NP hacia él

Un lenguaje que pertenece a NPC también pertenece a P , si $P = \text{NP}$. Ya que para una reducción polinomial, si el lenguaje de la derecha pertenece a P , entonces también lo hará el de la izquierda, y si un lenguaje pertenece a NPC, entonces está a la derecha (hay una reducción polinomial) de todo

lenguaje de NP, por lo que si se cumplen ambas condiciones para el mismo lenguaje (está en P y en NPC), entonces $P = NP$.

Y por el contrarrecíproco, si $P \neq NP$, un lenguaje NP-completo no pertenece a P

- d. *¿Enunciar el esquema visto en clase para agregar un lenguaje a la clase NPC*

Para agregar un lenguaje L a la clase NPC hay dos opciones:

- Encontrar una MT M_f que haga una reducción polinomial de todos los lenguajes $\in P$ hacia L
- Encontrar una MT M_f que haga una reducción polinomial de un lenguaje $L_{NPC} \in NPC$ hacia L

En ambos casos, como L es tan o más difícil que todos los lenguajes de NPC, entonces $L \in NPC$

- e. *¿Cuándo se sospecha que un lenguaje de NP está en NPI?*

Se sospecha que un lenguaje L está en NPI cuando no se encuentra un algoritmo que lo decida en tiempo $\text{poly}(n)$, y tampoco se encuentra alguna reducción polinomial de un lenguaje $\in NPC$ a L

Ejercicio 2

Probar:

Ayuda: recurrir directamente a la definición de NPC

- a. Si $L_1 \in NPC$ y $L_2 \in NPC$, entonces $L_1 \leq_p L_2$ y $L_2 \leq_p L_1$

Eso se cumple dado que si $L_1 \leq_p L_2$ entonces L_2 es tan o más difícil que L_1 , y si $L_2 \leq_p L_1$, entonces L_1 es tan o más difícil que L_2 y ambas cosas se cumplen ya que la hipótesis dice que tanto L_1 como L_2 pertenecen a NPC, es decir, son igual de difíciles

- b. Si $L_1 \leq_p L_2$, $L_2 \leq_p L_1$, y $L_1 \in NPC$, entonces $L_2 \in NPC$

Eso se cumple dado que si $L_1 \leq_p L_2$ entonces L_2 es tan o más difícil que L_1 , y si $L_2 \leq_p L_1$, entonces L_1 es tan o más difícil que L_2 , por lo que, si ambas cosas suceden (por la hipótesis), y L_1 pertenece a NPC, entonces, como L_1 debe ser tan o más difícil que L_2 y también al revés, L_2 es igual de difícil que L_1 , es decir, pertenece a NPC

Ejercicio 3

Un lenguaje es CO-NP-completo si y sólo si todos los lenguajes de CO-NP se reducen polinomialmente a él. Probar que SAT^C es CO-NP-completo

Ayuda: $L_1 \leq_p L_2$ si y sólo si $L_1^C \leq_p L_2^C$

SAT^C es CO-NP-completo ya que sabemos que existe una reducción polinomial de todo lenguaje perteneciente a NP hacia SAT (se vio en la teoría), y como $L_1 \leq_p L_2$ si y sólo si $L_1^C \leq_p L_2^C$, entonces, existe una reducción polinomial de todo lenguaje perteneciente a CO-NP hacia SAT^C

Ejercicio 4

Sean los lenguajes A y B , tales que $A \neq \emptyset$, $A \neq \Sigma^*$, y $B \in P$. Probar: $(A \cap B) \leq_p A$

Ayuda: intentar con una reducción polinomial que, dada una cadena w , lo primero que haga sea chequear si $w \in B$, teniendo en cuenta que existe un elemento e que no está en A

Habría que construir una MT M_f que realice una reducción de $(A \cap B)$ hacia A en tiempo $\text{poly}(|w|)$. Para esto, M_f al recibir una entrada w , chequea si pertenece a B (lo cual se hace en tiempo $\text{poly}(|w|)$ ya que B pertenece a P):

- Si $w \in B$, entonces M_f retorna w :
 - Si $w \in A \cap B$, y está en B , entonces también estará en A
 - Si $w \notin A \cap B$, y está en B , entonces no estará en A
- Si $w \notin B$, entonces M_f simplemente devuelve el elemento e que no está en A

Ejercicio 5

Sea el lenguaje $SH-s-t = \{(G, s, t) \mid G \text{ es un grafo no dirigido y tiene un camino de Hamilton del vértice } s \text{ al vértice } t\}$. Un grafo $G = (V, E)$ tiene un camino de Hamilton del vértice s al vértice t si y sólo si G tiene un camino entre s y t que recorre todos los vértices restantes una sola vez. Probar que $SH-s-t$ es NP-completo. Ayuda: se sabe que CH, el lenguaje correspondiente al problema del circuito hamiltoniano, es NP-completo

Queda pendiente (y nunca será hecho 😊)

Práctica 7

Ejercicio 1

Responder breve y claramente:

- a. *¿Por qué en la complejidad espacial se utilizan MT con una cinta de entrada de sólo lectura?*

Para evitar que el tamaño de la entrada influya en la complejidad espacial. De esta manera, se pueden tener algoritmos con espacio $\log(n)$

- b. *¿Por qué si una MT tarda tiempo $\text{poly}(n)$ entonces ocupa espacio $\text{poly}(n)$, y si ocupa espacio $\text{poly}(n)$ puede llegar a tardar tiempo $\exp(n)$?*

Si una MT tarda tiempo $\text{poly}(n)$ entonces ocupa espacio $\text{poly}(n)$ porque al hacer a lo sumo $\text{poly}(n)$ pasos, solo podrá ocupar como máximo $\text{poly}(n)$ celdas, si se moviera siempre en la misma dirección

Por otro lado, si una MT ocupa espacio $\text{poly}(n)$, entonces tendrá una cantidad de configuraciones que se ve elevada al tamaño de la entrada: $c^{S(n)}$, donde c es una constante y $S(n)$ el tamaño máximo de celdas en base al tamaño de la entrada (n) (en este caso $S(n)$ es $\text{poly}(n)$), por lo que podría tardar tiempo $\exp(n)$: $c^{S(n)} = c^{\text{poly}(n)} = \exp(n)$

- c. *¿Por qué los lenguajes de la clase LOGSPACE son tratables?*

Porque si una MT ocupa espacio $\log_k(n)$ tendrá una cantidad de configuraciones que se ve elevada al tamaño de la entrada $c^{\log_k(n)}$, donde c es una constante, por lo que simplificando queda $n^{\log_k(c)}$, lo cual es de orden $\text{poly}(n)$, por lo que cualquier lenguaje que esté en LOGSPACE estará en P, y por ende, será tratable

Ejercicio 2

Describir la idea general de una MT M que decida el lenguaje $L = \{a^n b^n \mid n \geq 1\}$ en espacio logarítmico

Ayuda: basarse en el ejemplo mostrado en clase.

Se debería definir una MT M que haga:

1. En la cinta 1 (la de la entrada sería la 0) coloca un contador i , que inicializa en 1
2. En la cinta 2 coloca un contador j , que inicializa en $|w|$, si es impar rechaza
3. En la cinta 3 se coloca el símbolo w_i
4. En la cinta 4 se coloca el símbolo w_j
5.
 - a. Si $i > j$, entonces acepta
 - b. Si $i < j$, entonces el símbolo de la cinta 3, debe ser a y el de la cinta 4 debe ser b , sino rechaza
6. $i++$, $j--$

7. Vuelve al paso 3

Esta MT ocupa tanto espacio como lo haga la cinta 2, ya que la cinta 3 y 4 ocupan una celda, y la cinta 1 ocupará menos que la cinta 2 (al ser i menor que j), y la cinta 2 ocupará tanto espacio como lo haga la codificación binaria (o n -aria con $n > 1$) de $|w|$, es decir $\log_2(|w|)$ (o $\log_n(|w|)$ con $n > 1$), por lo que L pertenece a LOGSPACE

Ejercicio 3

Describir la idea general de una MT M que decida el lenguaje SAT en espacio polinomial

Ayuda: la generación y la evaluación de una asignación de valores de verdad se pueden efectuar en tiempo polinomial

La MT haría:

1. En la cinta 1 se generan todas las asignación de valores de verdad, pero de una a la vez, lo cual ocupará un espacio de m donde m es la cantidad de variables que tiene la fórmula booleana (cada valor de verdad para cada variable se asigna en 1 bit), y $m = O(n) = \text{poly}(n)$
2. Se evalúa una a una cada asignación de valores de verdad usando la cinta 2, lo cual ocupa un espacio de $O(n)$, ya que para evaluar una asignación basta con ir paso a paso evaluando cada par de variables (teniendo en cuenta la precedencia de operadores), pero nunca superará un espacio de $O(n)$
 - a. Si alguna asignación de valores de verdad es verdadera, entonces se acepta
 - b. Si al finalizar ninguna asignación de valores de verdad es verdadera, entonces se rechaza

Ejercicio 4

Justificar por qué el lenguaje QSAT no pertenecería a P ni a NP

Ayuda: ¿qué forma tienen los certificados asociados a QSAT?

El lenguaje QSAT no estaría ni en P ni en NP porque sus certificados no serían sucintos al tener forma de árbol, lo cual crece exponencialmente con el tamaño de la entrada

Ejercicio 5

Probar que $NP \subseteq PSPACE$

Ayuda: Si L pertenece a NP , entonces existe una MT M_1 capaz de verificar en tiempo $\text{poly}(|w|)$ si una cadena w pertenece a L , con la ayuda de un certificado x de tamaño $\text{poly}(|w|)$. De esta manera, existe también una MT M_2 que decide L en espacio $\text{poly}(|w|)$, sin la ayuda de ningún certificado

Esto es así si la MT M_2 trabaja de la siguiente manera:

1. En la cinta 1, se generarán todos los posibles certificados x (los cuales son de tamaño $\text{poly}(|w|)$, sino L no estaría en NP), pero se hará dicha generación de a 1 certificado
2. Para cada certificado generado, se lo verificará en la cinta 2, lo cual sabemos que se hace en tiempo $\text{poly}(|w|)$, por lo que también se hará en espacio $\text{poly}(|w|)$
3.
 - a. Si para algún certificado se define que w pertenece a L , entonces se acepta
 - b. Si para ningún certificado se define que w pertenece a L , entonces se rechaza

Ejercicio 6

Supongamos que existe una MT M de tiempo polinomial que, dado un grafo G , devuelve un circuito de Hamilton de G si existe o responde no si no existe. Describir la idea general de una MT M' que, utilizando M , decida en tiempo polinomial si un grafo G tiene un circuito de Hamilton

Ayuda: basarse en el ejemplo mostrado en clase con FSAT y SAT

La MT M' debería:

1. Ejecutar M con G (tiempo polinomial)
 - a. Si encuentra algún circuito de Hamilton en G , M' acepta
 - b. Si responde que no, M' rechaza

El tiempo de M' es $\text{poly}(n)$ porque la ejecución de M es $\text{poly}(n)$

Ejercicio 7

Sea la MTP M que definimos en clase para decidir probabilísticamente el lenguaje $\text{MAYSAT} = \{\varphi \mid \varphi \text{ es una fórmula booleana satisfactible por más de la mitad de las posibles asignaciones de verdad}\}$. Hemos indicado que para toda φ , si $\varphi \in \text{MAYSAT}$ entonces M la acepta con probabilidad $> \frac{1}{2}$, y si $\varphi \notin \text{MAYSAT}$ entonces M la rechaza con probabilidad $\frac{1}{2}$. Precizando más la primera probabilidad: asumiendo que M tarda $p(n)$, si $\varphi \in \text{MAYSAT}$ entonces M la acepta con probabilidad $\frac{1}{2} + \frac{1}{2^{p(n)}}$. Explicar por qué

Ayuda: en tiempo $p(n)$, M puede producir $2^{p(n)}$ computaciones posibles, y entonces, ¿cuántas son de aceptación como mínimo si $\varphi \in \text{MAYSAT}$?

Queda pendiente (tampoco será hecho)

Ejercicio 8

Considerando el ejemplo de computación cuántica mostrado en clase, indicar los resultados posibles cuando en lugar de arrancar con el estado inicial 00 , la computación arranca con:

a. *El estado inicial 01*

- Inicio: $|01\rangle$
- Hadamard sobre el primer cúbit: superposición de $|01\rangle$ y $|11\rangle$
- CNOT sobre los dos cubits: superposición de $|01\rangle$ y $|10\rangle$

b. *El estado inicial 10*

- Inicio: $|10\rangle$
- Hadamard sobre el primer cúbit: superposición de $|10\rangle$ y $|00\rangle$
- CNOT sobre los dos cubits: superposición de $|11\rangle$ y $|00\rangle$

c. *El estado inicial 11*

- Inicio: $|11\rangle$
- Hadamard sobre el primer cúbit: superposición de $|11\rangle$ y $|01\rangle$
- CNOT sobre los dos cubits: superposición de $|10\rangle$ y $|01\rangle$